

MVP End-to-End Master Command

Thirteen front-end commands to make the version we already have work, end to end, for a real customer — plus the backend blockers that no amount of front-end work can move.

Field	Value
Prepared	19 August 2026 . Every figure marked <i>measured</i> was taken on this date against <code>api.servana.com.ph</code> or the repository itself.
Repository	<code>C:\Users\paulg\OneDrive\Desktop\servana_client-unified</code> — the unified working copy. Supersedes <code>servana_client-mobile</code> .
Remote	<code>github.com/Upupapp/ServanaClientAPP</code> . Nothing in this document has been pushed.
Gates today	All green — <code>dart format</code> exit 0 (729 files, 0 changed); <code>flutter analyze</code> No issues found ; <code>flutter test</code> 2161 passed , 6 skipped.
Scope	Front-end only . Backend items appear as blockers and evidence, never as tasks in a TAB.
Definition of MVP	One customer can install the app, create an account, sign in, find a service, book it, pay for it, watch it happen, talk to the provider, and review it — without hitting a screen that lies, dead-ends, or cannot be completed.

Read this first. As of this document, production is answering `500` on **every database-backed route**. The customer app cannot browse, search, sign in or book — on either transport. The cause is a single broken database credential in the running API process, and it is **backend work**. Until it is fixed, no front-end TAB below can be verified end to end. See **Section B**.

Front-end commands are ordered so that each one is verifiable the moment the backend is healthy. TAB 01 exists to make “works end to end” a measurement rather than an opinion.

Section A — What is already true

CLOSED TODAY, IN THIS REPOSITORY

Commit	What
e29fcd9	Unified two divergent <code>main</code> branches — 31 production-launch commits on GitHub and 43 front-end SWEEP commits locally that had never seen each other.
00320db	Two login-failure assertions that could not fail (the analyzer rejected them as tautologies) now classify through the base type, as production does.
1bd9e39	Analyzer taken back to zero issues across the merged tree.
7c4a178	The app no longer tells customers their password is wrong when the server is down. See TAB 02.

FRONT-END AREAS MEASURED HEALTHY — DO NOT SPEND MVP TIME HERE

- **Token refresh.** Single-flight (`_refreshInFlight`), one owner for sign-out, 401 handled in one place. No logout loop of the kind the provider portal has.
- **Catalog to booking handoff.** `canonical_booking_handoff.dart` routes a canonical Service into a real checkout, and an **unknown category falls to the generic flow rather than failing** — a new category is bookable the day an admin creates it, with no app release.
- **Booking submission error copy.** Substring-based, but its default is `serverFailure` and reads “Something went wrong. Please try again.” It fails safe; it never blames the customer.
- **Route reachability.** 67 `GoRoute` s, derived from screen constants rather than string literals.
- **Capability gating.** Deny-by-default, build-time only, not remote-configurable. All 15 capabilities are OFF, so the shipped app is 100% legacy transport. **For MVP this is correct — leave it alone** (TAB 13).

Section B — Backend blockers (not front-end work)

These are recorded because they determine whether the front end can be verified at all. **No TAB in Section C asks anyone to fix these.**

B1 — P0, LIVE NOW: THE PRODUCTION DATABASE CREDENTIAL IS BROKEN

Measured 19 Aug 2026 against `api.servana.com.ph` :

```
GET /healthz          200 {"status":"alive"}
GET /readyz           503 "phase":"degraded", ready:false
GET /api/catalog       500 {"status":"failed","message":"Server error"}
GET /api/catalog/summary 500
GET /api/catalog/services 500
GET /api/services      500
GET /api/services/full 500
GET /api/v1/catalog    500 <-- v1 IS deployed; it fails the same way
GET /api/categories    401 <-- auth middleware rejects before the DB
```

All four readiness dependencies fail with one error:

```
SASL: SCRAM-SERVER-FIRST-MESSAGE: client password must be a string
```

That is the `pg` driver receiving `undefined` as the password — the database password is not reaching the running process. `uptimeMs` was about 2.3 hours, so the process restarted and came up without it.

Consequence: every database-backed route returns 500. Public reads fail outright; authenticated routes return 401 only because the auth middleware rejects them before they touch the database. **The app is unusable in production right now.**

Note for whoever fixes it: this service has been bitten before by having **two** env files, with `pm2` env showing neither. Check what the running process actually has, not what the file says.

B2 — VERIFY AFTER B1: THE CATALOG ROUTE SHADOWING

On 18 August `GET /api/catalog` answered **401** because a booking wildcard `GET /:id` was mounted ahead of it. Today it answers **500**, which is the database fault masking whatever routing state exists underneath. **Re-measure all three catalog routes once B1 is fixed** — if they return 401 again rather than 200, the mount-order fix is still undeployed. The Home “More services” row and the whole Search feature depend on it.

B3 — COVERAGE AND BRANCH SELECTION ARE KEYED TO LEGACY FAMILY IDS

`GET /api/services/:familyId/branches` and `/coverage-geo` take a legacy family id. A canonical Service created through the Admin API has no legacy option, so neither is reachable for it. The front end can only degrade honestly here (TAB 07); making them work is backend scope.

B4 — THERE IS NO CANONICAL BOOKING CREATE

`POST /api/bookings` is classified `KEEP` with no `/api/v1` successor and none planned. This is why the capability enum has `bookingReads` and `bookingLifecycle` but deliberately no `booking`. It does not block MVP — the legacy write is what ships — but it does block any future claim that the booking domain has migrated.

B5 — VERIFYSMOBILE NEEDS A FIREBASE PHONE CREDENTIAL

The canonical endpoint takes `{idToken}`, not `{mobileNumber, otp}`. That is a redesign, not a rename. It is why mobile-number login is still “coming soon” in the UI (TAB 04).

B6 — DISPUTES ARE ADMIN-ONLY

The only legacy dispute route is admin-scoped, so `canOpenDispute` is false for every customer. The controller already exposes the capability; the screen appears the day a transport can serve one. Out of MVP scope.

Section C — The front-end commands

One TAB per command. Each is independently verifiable. **Every TAB ends with evidence, not an assertion.**

TAB	Command	Blocked by	MVP
01	Make “works end to end” a measurement	B1 to run it	Required
02	Finish the sign-in and registration honesty pass	—	Required
03	Prove the account-creation loop closes	B1	Required
04	Remove every affordance that cannot work	—	Required
05	Home and Search must say what is actually wrong	B1, B2	Required
06	Booking creation across every production category	B1	Required
07	Address, schedule and coverage degrade honestly	B3	Required
08	Payment: cash and PayMongo, including the unhappy paths	B1	Required
09	My Bookings, detail, tracking and OTP completion	B1	Required
10	Messaging the provider	B1	Required
11	Review after completion	B1	Required
12	Session resilience and the offline banner truthfulness	—	Required
13	Cut the MVP build with capabilities OFF	01–12	Required

TAB 01 — Make “works end to end” a measurement

WHY

“The app works” is currently an opinion held by whoever last tapped through it. The suite has 2161 tests and none of them walks a customer from install to review, because every one of them stops at a seam. That is the right design for unit tests and the wrong basis for an MVP go/no-go.

DO

- Write the ten-step customer journey down as an explicit, ordered checklist — install, create account, verify, sign in, browse, open a service, book, pay, track, message, review.
- Build an `integration_test/` walkthrough that drives as much of it as a device can without a live provider on the other end. The existing `test/support/screen_test_container.dart` already registers the real DI graph over a `MockClient`; extend that pattern rather than inventing a second one.
- For the steps a test cannot reach (a provider accepting, an OTP handed over in person), write the manual script with the exact account, exact service and exact expected screen at each step.

DONE WHEN

A single command produces a pass or fail for the automatable portion, and a one-page manual script exists for the rest, with a named test account. **Record which steps are automated and which are manual** — a walkthrough that silently skips payment reads as a green light for payment.

TAB 02 — Finish the sign-in and registration honesty pass

Largely closed today in 7c4a178 . This TAB is the remainder.

WHAT WAS WRONG, AND WHY IT MATTERED

`HttpBackend` turns an unparseable response body into `Login failed (502)`. and a JSON body with no message into `Login failed.` — and `ErrorMessageMapper` classified login failures by searching those strings for keywords. Neither `502` nor `504` appears in any list; only `500` and `503` do. So both strings fell through every branch onto the default:

```
"The email or password is incorrect."
```

An nginx gateway error in front of a restarting API — **exactly today's production condition** — told the customer their password was wrong. A customer who reads that changes a password that was never wrong, and support cannot reproduce it.

The status code is now threaded through `Backend.authenticate` and `registerCustomer` and consulted **first**. `401` and `403` still defer to the body, because the difference between an unverified account and a disabled one lives there and both are `401`.

Registration had the mirror fault: `registration_bloc` put the raw backend string on screen and showed `e.toString()` for an exception, while `ErrorMessageMapper.forRegistration` — written for exactly that, and carrying the rule *no raw backend strings or stack traces should ever reach the UI* — had **no callers at all**.

22 tests were added and **mutation-checked**: with the status branch disabled, 15 of them fail.

REMAINING WORK

- Apply the same status-first rule to `resendVerificationEmail`, which still returns a bare record with no status and sits on the verification loop TAB 03 depends on.
- Add a **widget-level** test that the sign-in screen renders the mapped copy. The current tests pin the mapper and the bloc; nothing asserts what the customer actually sees.
- Audit the four other sites that render connectivity copy without consulting `isTransport`: `address_repository.dart:127`, `booking_submission_result.dart:124`, and `notifications_screen.dart:154,196` (a wifi-off icon). A `500` is not a wifi problem.

TAB 03 — Prove the account-creation loop closes

WHY

Sign-up is the only door into the app, and it is a loop, not a step: register, mail lands, OTP or link, verified, sign in. Any break makes the app unusable for a brand-new customer, which is the only kind of customer an MVP has.

DO

- Walk the whole loop with a fresh address. Confirm the verification mail actually arrives, and confirm resend works and is rate-limited sanely.
- Confirm the app bodies match what the deployed handler accepts. The v1 request schemas are **not enforced at runtime** — `src/api/v1/register.ts` runs no validator, so the handler is the authority and it accepts legacy aliases. Do not read the schema and assume.
- Confirm a customer who abandons mid-verification can get back in rather than being stranded between states.

DONE WHEN

A brand-new address reaches a signed-in Home screen, and the exact bodies sent at each step are recorded in the TAB.

TAB 04 — Remove every affordance that cannot work

WHY

An MVP is judged by what a customer can finish. A control that is visible and cannot work costs more trust than a feature that is absent — the customer does not know it is unbuilt, so they read it as broken.

KNOWN, MEASURED

Affordance	State	MVP call
Mobile-number login	Says “coming soon”. Cannot work: the canonical endpoint needs a Firebase phone credential (B5).	Remove for MVP , ship email-only
Rewards screen	Placeholder — “Rewards coming soon”.	Remove from the drawer
Favourites	Placeholder, same pattern.	Remove from the drawer
Disputes	No screen. <code>canOpenDispute</code> false everywhere (B6).	Correctly absent — leave

DO

Sweep every screen for a control that cannot complete, and either remove it or make the disabled state say *why*. Then add a test that asserts the property rather than the instances — a test written for one orphan tends to find the next one immediately.

TAB 05 — Home and Search must say what is actually wrong

WHY

The catalog is 500ing (B1 and B2), so this is not hypothetical: it is what every customer sees today. The Home “More services” row draws nothing and Search returns nothing.

The fail-safe is working as designed — the curated four cards still render and nothing crashes. What must be true is that the customer is **told the truth**: the service listing is unavailable, not “check your connection”.

DO

- Confirm the honesty rule from TAB 02 reaches Home and Search, not just sign-in. `home_composition.dart:128` already consults `isTransport`; verify every other failure surface does.
- Confirm an absent section and a failed section stay visibly different. There is nothing to retry on an absent one, so a retry button there is a lie.
- **Do not** feed catalog categories into the Home curated grid. Home keys use underscores (`beauty_wellness`) and drive both navigation and the campaign registry; the backend slugify produces hyphens. A campaign registered against a key the catalog does not use returns null: no popup, no exception, tests pass. The catalog must only *add* what the curated four do not cover, routed by `catalog_categories.id`.

TAB 06 — Booking creation across every production category

WHY

This is the transaction. Everything before it is browsing.

The handoff resolves a canonical Service into one of two checkouts by category id (2 goes to aircon, everything else to the generic Beauty and Wellness flow). That default is deliberate and good. It has not been proven for all three production categories.

DO

- Book one service in each production category — 1 Home Maintenance, 2 Home Services, 3 Personal Care.
- Book a Service **created through the Admin API**, which has no legacy option row. This is the case where `services.id == service_options.id` stops holding, and it is the one that will break first in production.
- Confirm the payload `serviceOptionId` is the canonical `services.id` the customer actually selected, and that add-on selection does not follow the customer to the next service.

TAB 07 — Address, schedule and coverage degrade honestly

WHY

Branch and slot selection and coverage-geo are keyed to legacy family ids and are **unreachable for a canonical Service** (B3). The front end cannot fix that. What it must not do is present a coverage answer it did not get.

DO

- Decide and implement the MVP behaviour when coverage cannot be determined. An undeterminable coverage answer should **deny or ask**, never silently allow — a booking accepted outside the service area costs a real dispatch.
- Confirm saved addresses work: add, select, make primary.
- Confirm the Manila coordinate fallback is not silently applied to a booking whose real coordinates are simply absent from the response.

TAB 08 — Payment: cash and PayMongo, including the unhappy paths

WHY

Payment is where an MVP either earns money or loses trust. The client sends `CASH` or `PAYMONGO` and the backend types the column `CASH|GCASH|PAYMONGO`.

DO

- Complete a cash booking end to end.
- Complete a PayMongo booking through `PaymentWebViewScreen`, then deliberately **cancel** one and **abandon** one (close the sheet mid-flow).
- Confirm each of those three outcomes leaves a booking in a state the customer can act on. An abandoned payment that leaves an invisible half-booking is worse than a failed one.
- Confirm the amount shown at checkout is the amount charged, and that a price arriving as a JSON string rather than a number still renders. Postgres numeric arrives as int, double or string depending on value and driver.

TAB 09 — My Bookings, detail, tracking and OTP completion

DO

- Confirm a booking created in TAB 06 appears in My Bookings immediately, with the right total — not ₱0.00.
- Confirm booking detail reads through `BookingRepository` and shows status, schedule, address, provider and total.
- Confirm tracking renders when the provider is en route, and degrades honestly when there is no location rather than dropping a pin on a default coordinate.
- Confirm the completion OTP path works from the customer side.

Watch for: the domain model was once lossier than the screen own parsing — a blind swap onto the repository regressed five behaviours including the total. Pin the model against the screen before trusting it.

TAB 10 — Messaging the provider

DO

- Open the conversation from a booking, send and receive, confirm unread counts and read receipts settle.
- Confirm the conversation is reachable from both the booking detail and the inbox, and that both resolve to the same thread.
- Confirm a message that fails to send is visibly unsent and retryable, rather than silently dropped.

TAB 11 — Review after completion

DO

- Confirm review eligibility appears only after completion, and that submitting once closes the affordance.
- Confirm the rating reaches the provider aggregate the app reads back.

Note: only 4 of 9 review calls have a canonical successor, which is why `reviews` is absent from the capability enum. Irrelevant for MVP — the legacy path is what ships.

TAB 12 — Session resilience and the offline banner truthfulness

DO

- Confirm a token expiring mid-session refreshes once, not once per in-flight request, and that a dead refresh token signs the customer out exactly once.
- Confirm the offline banner appears only when the device is genuinely offline. During B1 the device is online and the server is broken — the banner must not claim otherwise.
- Confirm backgrounding the app for an hour and returning does not strand the customer on a blank screen.

TAB 13 — Cut the MVP build with capabilities OFF

WHY OFF IS THE RIGHT CALL

`/api/v1` is deployed — measured today, `GET /api/v1/catalog` returns 500 rather than 404, which means the route exists and reached the database. So the gate original premise (“v1 is unpushed”) is now out of date.

That is **not** a reason to enable it for MVP. Both transports fail identically today, so switching buys nothing, and every capability flipped on is a surface that has never carried real customer traffic. **Ship the transport that ships.** Migrating is a post-MVP command with its own evidence.

DO

- Cut a release build with all 15 capabilities off, and assert that in a test rather than trusting the default.
- Run the built artefact, not the debug one. R8, signing and the version floors are already done; confirm the shrunk build still runs the journey from TAB 01.
- Confirm the version gate can retire this build later, and that the deep links resolve.

Section D — How to run this

1. **Get B1 fixed.** Nothing else can be verified until the API can reach its database. This is minutes of work for whoever holds the server, and it is currently costing every customer the whole app.
2. **Re-measure B2** the moment B1 is green. If the catalog routes answer 401 rather than 200, the mount-order fix is undeployed and TAB 05 is blocked.
3. **Run TAB 01** to fix what “end to end” means, then TABs 02 to 04, which need no backend.
4. **Then TABs 05 to 11 in order** — they follow the customer own path, so a break in one is felt by the next.
5. **TAB 12 and 13 last**, because they are about the build rather than the journey.

Standing rule for every TAB: finish with evidence — the command run, the status codes seen, the screen reached. A TAB that ends in an assertion has not been done. Where a gate is added, watch it fail once before believing it passes.